

Inferenza di una rete neurale nel kernel Linux

Le slide una alla volta, ognuna con la sua spiegazione

Ogni pagina ha l'immagine di una slide e sotto la spiegazione. I riquadri grigi sono **esempi con i numeri**; i riquadri gialli segnalano i **punti da correggere o da dire con cautela**; i riquadri verdi le **verifiche** fatte ricontrollando i dati.

Le parole e le unità che tornano sempre

Nodo	Il computer che riceve il pacchetto, esegue la rete neurale e lo inoltra. Nel banco è il core 6.
XDP	Il primo punto del kernel Linux in cui un programma può vedere un pacchetto appena arrivato.
Pipeline	Una delle versioni del programma che esegue la rete neurale (P1, P1.5, P2, P3).
baseline / rxonly	I due programmi di riferimento: la baseline inoltra senza rete neurale, rxonly riceve e basta.
Coda	I 256 posti davanti al nodo dove i pacchetti aspettano. Se è piena, chi arriva è respinto.
Coda d'uscita	La coda del cavo virtuale verso il nodo successivo. Con più core sul nodo ce n'è una per core, svuotata da un core suo.
Capacità	Quanti pacchetti al secondo il nodo riesce a elaborare al massimo.
Collo di bottiglia	Il pezzo più lento della catena: decide quanto traffico passa.
M/s	Milioni di pacchetti al secondo.
ns, μ s, ms	Nanosecondi (miliardesimi di secondo), microsecondi ($1\ \mu\text{s} = 1\ 000\ \text{ns}$), millisecondi ($1\ \text{ms} = 1\ 000\ \mu\text{s}$).
Gbit/s	Miliardi di bit al secondo. Con pacchetti da 64 byte, $1\ \text{Gbit/s} \approx 1,95$ milioni di pacchetti al secondo.
Costo per pacchetto	1 diviso la capacità. 2 milioni di pacchetti al secondo = 500 ns per pacchetto.
Tempo di CPU per pacchetto	Quanto tempo il core del nodo lavora davvero per ogni pacchetto: occupazione del core $\times$ 1 diviso la capacità. Coincide con il costo per pacchetto quando il nodo lavora il 100% del tempo.
Scrittore	Chi mette pacchetti in una coda. Con una scheda di rete vera è la scheda stessa: uno per coda.

P-core, E-core, LP E-core	I tre tipi di core del processore: veloci (Redwood Cove), efficienti (Crestmont, in moduli da quattro che dividono cache L2 e frequenza) e a basso consumo (fuori dalla cache L3, al massimo 2,5 GHz).
Istruzioni per pacchetto	Quante istruzioni macchina il core del nodo esegue per ogni pacchetto, contate dal processore stesso (contatori hardware). Dipendono dal codice, non dal core.
Ciclo, IPC	Il ciclo è il battito dell'orologio del core (3,5 miliardi al secondo a 3,5 GHz). IPC = istruzioni eseguite per ciclo: dice quanto lavoro il core fa a ogni battito, e dipende dal core.
BPF_PROG_TEST_RUN	Una funzione del kernel che esegue il programma XDP su un pacchetto preparato, in un ciclo, senza traffico vero: misura il costo del solo programma.

TESI · EBPF / XDP

Inferenza di una rete neurale nel kernel Linux con eBPF/XDP

Quattro versioni della stessa rete neurale, misurate sul traffico vero: quanto costa un pacchetto, quante istruzioni esegue, e che cosa cambia da un P-core a un E-core.

Ettore Cantile
ottobre 2026

La tesi esegue una piccola rete neurale **dentro il kernel Linux**, nel punto in cui il pacchetto è appena arrivato (XDP), per decidere da quale porta farlo uscire. La stessa rete è scritta in quattro modi diversi, da completamente compilata a completamente configurabile, e la domanda è quanto costa, per ogni pacchetto, ciascuna scelta.

La presentazione ha due parti:

1. **Il nodo sotto traffico vero** (slide 3–15): un generatore manda pacchetti veri al nodo e si misura quanti ne regge, quante istruzioni esegue per ognuno, e che cosa cambia se il nodo gira su un core veloce (P-core), efficiente (E-core) o a basso consumo (LP E-core). È la parte con le misure nuove.
2. **Il programma da solo** (slide 16–22): lo stesso programma eseguito dal kernel su un pacchetto preparato, senza traffico, per vedere come il costo cresce con la forma della rete.

Tutte le cifre vengono dalla campagna del 6 ottobre 2026 sul portatile di laboratorio (Intel Core Ultra 7 155H, Ubuntu 24.04), con i core del banco a frequenza fissa: 3,5 GHz per P-core ed E-core, 2,5 GHz per il LP E-core.

Le quattro versioni

LE QUATTRO VERSIONI

Stessa rete, quattro modi di scriverla

Tutte e quattro calcolano la stessa rete e prendono la stessa decisione. Cambia solo **quanto sanno già** quando vengono compilate.

Versione	Nodo	Porte del nodo	Pesi	Forma della rete	Profondità
P1 · p1_static	compilato	compilabili *	compilati	compilata	compilata
P1.5 · hardcoded	da tabella	tutte previste	compilati	compilata	compilata
P2 · template	da tabella	tutte previste	da tabella	entro limiti fissi	compilata
P3 · modular	da tabella	tutte previste	da tabella	da tabella	da tabella

* Solo P1 statica può congelare anche **quali interfacce il nodo ha davvero**: le colonne dei collegamenti che non esistono non vengono generate (se un link è su o giù resta letto a runtime). Serve il nodo congelato, perché le porte dipendono da quale nodo si è. Nelle misure è usato solo nell'asse «porte presenti»; altrove P1 statica congela **solo il nodo**.

Più informazioni spostato dalla compilazione all'esecuzione, più il programma diventa **riutilizzabile** — e più cresce il **costo per pacchetto**.

Arancione = già noto quando si compila · verde = letto mentre il nodo gira

2

Tutte e quattro calcolano **la stessa rete** (65 ingressi, due strati nascosti da 4 neuroni, 7 uscite, 319 pesi) e prendono **la stessa decisione** sullo stesso pacchetto: lo verificano i test di correttezza. Cambia solo che cosa è già scritto nel programma quando lo si compila (arancione) e che cosa invece il programma legge da tabelle mentre gira (verde).

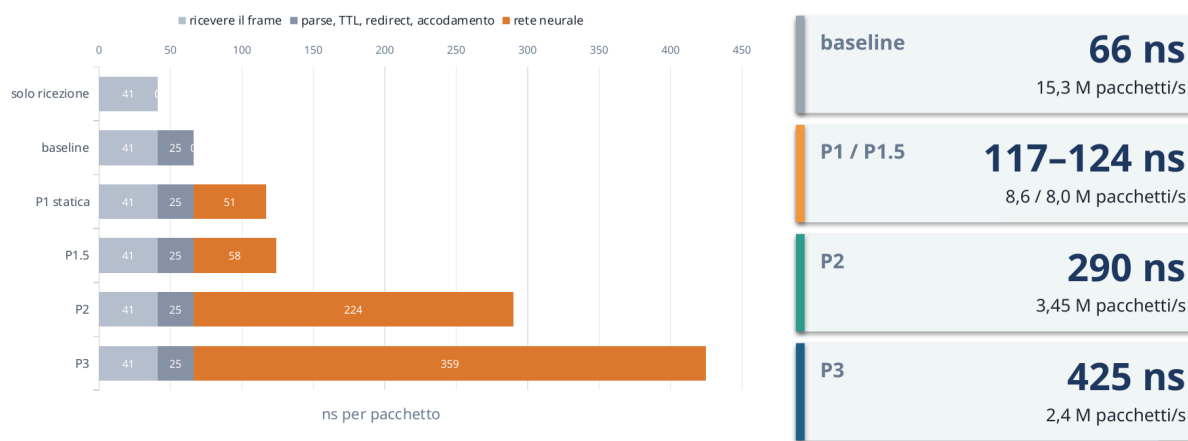
- **P1 statica**: pesi e identità del nodo scritti nel programma. Un binario per ogni nodo.
- **P1.5 hardcoded**: pesi nel programma, nodo letto da tabella. Un binario per modello.
- **P2 template**: anche la forma della rete si legge da tabella; nel programma ci sono solo i valori massimi ammessi. Un programma per molti modelli.
- **P3 modular**: anche il numero di strati si legge da tabella, e ogni strato è un programma a parte raggiunto con un salto. Un programma per tutto.

La regola che le slide successive misurano: più informazioni si spostano dalla compilazione all'esecuzione, più il programma diventa riutilizzabile, e più cresce il costo per pacchetto. I colori delle pipeline (P1 rosso, P1.5 arancione, P2 verde, P3 blu) restano gli stessi in tutti i grafici.

Quanto costa un pacchetto al nodo

IL NODO SOTTO TRAFFICO · P-CORE

Quanto costa un pacchetto al nodo



QUINDI Ricevere e inoltrare costano ~66 ns. La rete neurale aggiunge ~50-60 ns con i pesi compilati, ~225 ns se la forma è letta da tabella, ~360 ns se anche la profondità: un nodo regge da 8,6 a 2,4 milioni di pacchetti al secondo per core, e due core il 95-100 % del doppio.

3 giri, 64 byte, nodo su un P-core a 3,5 GHz, controllo di validità superato · stessa sessione per tutte le righe

3

Come si misura. Un generatore (xdp_gen) manda al nodo più pacchetti di quanti ne possa elaborare. La coda davanti al nodo resta sempre piena, il core del nodo lavora il 100% del tempo, e il ritmo a cui elabora è la sua capacità. Il costo per pacchetto è 1 diviso la capacità, ed è anche il tempo di CPU per pacchetto.

Le barre scompongono il costo in tre pezzi:

- **ricevere il frame** (grigio chiaro): quanto costa rxonly, il programma che riceve e butta, circa 41 ns;
- **parse, TTL, redirect, accodamento**: quello che la baseline fa in più (leggere l'intestazione, decrementare il TTL, inoltrare), 25 ns;
- **rete neurale** (arancione): quello che ogni pipeline aggiunge sopra la baseline.

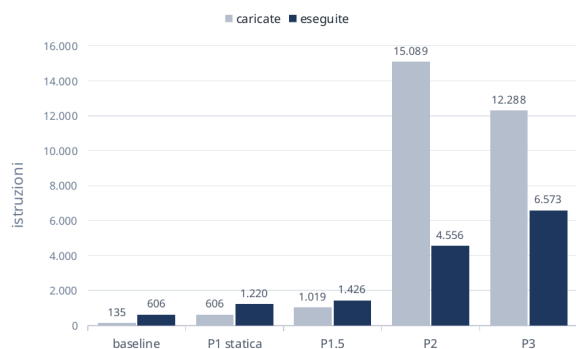
Esempio di calcolo. P2 regge 3,45 milioni di pacchetti al secondo: spende $1 / 3\,450\,000 \text{ s} = 290 \text{ ns}$ per pacchetto. La baseline ne spende 66. La differenza, 224 ns, è il tempo che P2 dedica alla rete neurale.

La rete neurale costa circa 50-60 ns con i pesi scritti nel programma (P1, P1.5), 224 ns con la forma letta da tabella (P2), 359 ns con anche la profondità (P3). Un nodo regge da 8,6 a 2,4 milioni di pacchetti al secondo per core.

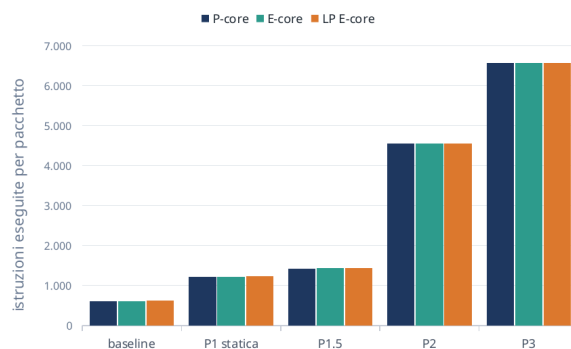
Le istruzioni: caricate contro eseguite

IL NUOVO ASSETTO · LE ISTRUZIONI

Quanto è grande il programma non dice quanto lavora



Caricate contro eseguite. P2 ne carica 15 089 e ne esegue 4 556; P3 ne carica meno (12 288) e ne esegue di più (6 573).



Stesse istruzioni su ogni core. Lo stesso codice: le istruzioni per pacchetto coincidono entro il 2%.

QUINDI La dimensione del programma misura quanto è **grande**, non quanto **lavora**: contano le istruzioni eseguite, comprese ricezione e redirect del kernel. Fra un core e l'altro le istruzioni non cambiano: cambia quante ne esegue il core per ciclo.

Caricate: BPF_PROG_TEST_RUN, istruzioni del programma · eseguite: contatori hardware del core del nodo sotto traffico, per pacchetto, nodo saturo

4

Grafico a sinistra. Due modi di contare le istruzioni:

- **caricate** (grigio): quante istruzioni ha il programma, contate dal kernel quando lo carica. È la dimensione del programma;
- **eseguite** (blu): quante istruzioni il core del nodo esegue davvero per ogni pacchetto, contate dal processore stesso con i contatori hardware durante la misura sotto traffico. Comprendono anche il lavoro del kernel attorno al programma (ricevere, inoltrare).

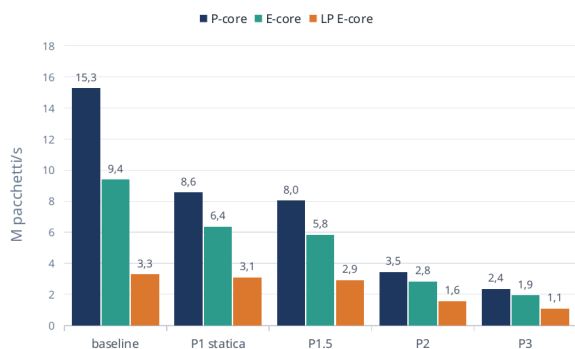
Le due misure dicono cose diverse. P2 è il programma più grande (15 089 istruzioni) ma ne esegue 4 556 per pacchetto: srotola cicli fino ai valori massimi, ma molti rami non vengono percorsi. P3 è più piccolo (12 288) ma ne esegue di più (6 573): rientra in sé stesso una volta per strato. **La dimensione non predice il lavoro.**

Grafico a destra. Le istruzioni eseguite per pacchetto sul P-core, sull'E-core e sul LP E-core: le tre barre sono uguali entro il 2%. Ovvio, perché il codice è lo stesso; ma è la base per la slide 6: se i core vanno a velocità diverse, non è perché eseguono istruzioni diverse.

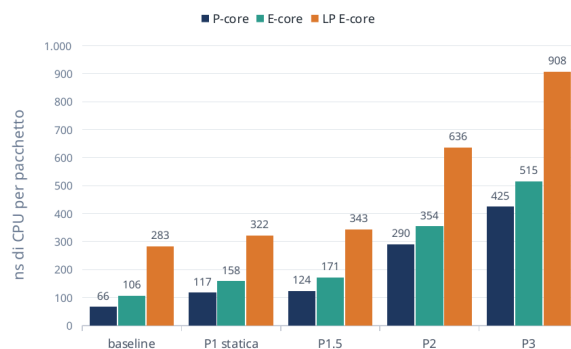
Il nuovo assetto: P-core, E-core, LP E-core

IL NUOVO ASSETTO · P-CORE, E-CORE, LP E-CORE

Su un E-core il nodo regge dal 61 all'82 % del P-core



Milioni di pacchetti al secondo. Baseline 15,3 → 9,4 → 3,3*; P3 2,4 → 1,9 → 1,1.



ns di CPU per pacchetto. Baseline +60% sull'E-core, l'inferenza di P3 solo +14%.

QUINDI

A parità di frequenza l'E-core è più lento soprattutto nel **lavoro di rete del kernel**: più la pipeline è pesante, meno pesa la differenza ($0,61 \times$ la baseline, $0,82 \times$ P2 e P3). Il LP E-core (fuori dalla L3, 2,5 GHz) regge il 36-47 %.

P-core e E-core a 3,5 GHz, LP E-core a 2,5 GHz; generatore e uscita su P-core · * LP E-core, baseline: nodo non saturo, limite inferiore

5

Il processore del portatile ha tre tipi di core: sei **P-core** veloci, otto **E-core** efficienti (in due moduli da quattro che dividono cache e frequenza) e due **LP E-core** a basso consumo, su un'altra parte del chip. La domanda: che cosa succede se il nodo gira su un core lento?

Il disegno della misura: cambia solo il core del nodo. Il generatore e il nodo successivo restano su P-core, così il generatore satura sempre il nodo. L'E-core gira alla stessa frequenza del P-core, 3,5 GHz, quindi la differenza dipende solo da come il core esegue il codice. Il LP E-core gira al suo massimo, 2,5 GHz. Gli altri tre core del modulo dell'E-core restano a riposo.

A sinistra i milioni di pacchetti al secondo, **a destra** i nanosecondi per pacchetto. Sull'E-core il nodo regge dal 61% (baseline) all'82% (P2, P3) del P-core; sul LP E-core dal 36 al 47% sulle pipeline.

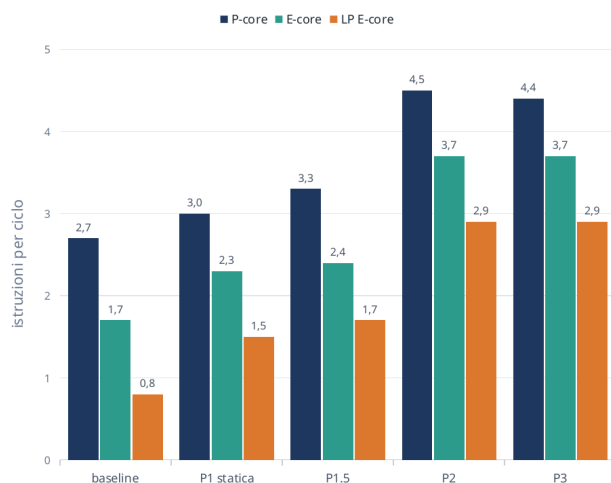
Esempio. P3: 2,36 milioni al secondo sul P-core, 1,94 sull'E-core (82%), 1,10 sul LP E-core (47%). In tempo: 425, 515 e 908 ns per pacchetto.

La baseline sul LP E-core. Sul LP E-core rxonly e baseline non saturano (nodo al 72% e al 93%): il generatore non riesce a offrire di più. Le loro cifre (asterisco) sono un limite inferiore.

Stesse istruzioni, meno istruzioni per ciclo

IL NUOVO ASSETTO · ISTRUZIONI PER CICLO

Stesse istruzioni, meno istruzioni per ciclo



Versione	Istruzioni	Cicli P	Cicli E
P1 statica	1 220	402	540
P1.5	1 426	432	587
P2	4 556	1 013	1 224
P3	6 573	1 483	1 785

QUINDI

Le **istruzioni per pacchetto** sono le stesse su ogni core: il codice non cambia. Cambia l'**IPC**: 2,7 → 1,7 per la baseline (lavoro di rete, salti, memoria), 4,4 → 3,7 per P3 (aritmetica lineare). Il LP E-core ha 14–19 mancate di cache per pacchetto.

Contatori hardware (perf_event_open) sul core del nodo, nella stessa finestra dei pacchetti · nodo saturo · istruzioni per ciclo nel grafico

6

Se le istruzioni per pacchetto sono le stesse (slide 4), la differenza fra i core sta nei **cicli**: quanti battiti d'orologio servono per eseguirle. Il rapporto fra istruzioni e cicli è l'**IPC**, istruzioni per ciclo.

Il grafico mostra l'IPC sui tre core; la tabella istruzioni e cicli per pacchetto su P-core ed E-core.

Esempio. P3 esegue 6 573 istruzioni per pacchetto. Il P-core ci mette 1 483 cicli (IPC 4,4), l'E-core 1 785 (IPC 3,7). A 3,5 GHz sono 424 e 510 ns: quasi esattamente i 425 e 515 ns misurati con il tempo di CPU, quindi i due metodi di misura concordano.

Due cose si leggono nel grafico:

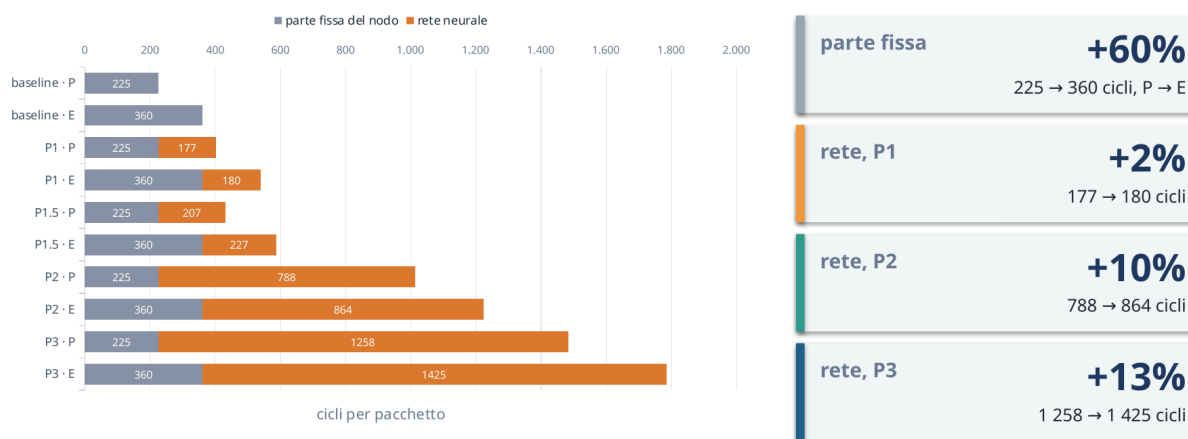
- l'IPC cresce con la pipeline: la baseline (lavoro del kernel, salti, accessi alla memoria) sta a 2,7 sul P-core, P2 e P3 (aritmetica in fila, senza bivi) oltre 4;
- il distacco fra i core è più grande dove l'IPC è basso: 2,7 → 1,7 per la baseline, 4,4 → 3,7 per P3.

Il LP E-core ha l'IPC più basso di tutti (0,8–2,9) perché, oltre al core più semplice, aspetta la memoria (slide 8).

Dove l'E-core perde tempo

IL NUOVO ASSETTO · DOVE L'E-CORE PERDE TEMPO

L'E-core rallenta il nodo, non la rete neurale



QUINDI

Ricevere, inoltrare e restituire la pagina è lavoro del kernel con salti e memoria: l'E-core lo esegue a **IPC 1,7 invece di 2,7**. L'aritmetica della rete è una fila di moltiplicazioni, e l'E-core la esegue **quasi altrettanto bene**.

Cicli per pacchetto dai contatori hardware, a 3,5 GHz · parte fissa = cicli della baseline, rete neurale = cicli della pipeline meno la baseline

7

Le barre dividono i cicli per pacchetto in due parti, sul P-core (P) e sull'E-core (E):

- **parte fissa del nodo** (grigio): i cicli della baseline, cioè ricevere, leggere l'intestazione, inoltrare, restituire la memoria del pacchetto;
- **rete neurale** (arancione): i cicli che la pipeline aggiunge sopra la baseline.

Il risultato è netto. La parte fissa passa da 225 a 360 cicli (+60%). La rete neurale cambia poco: P1 177 → 180 cicli (+2%), P2 788 → 864 (+10%), P3 1 258 → 1 425 (+13%).

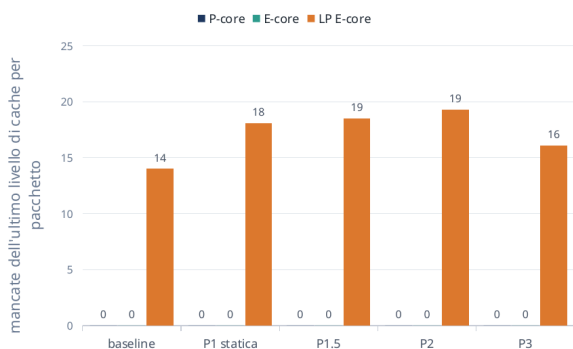
Perché. La parte fissa è codice del kernel con molti salti e accessi alla memoria, dove un core più semplice si ferma spesso ad aspettare. La rete neurale è una fila di moltiplicazioni e somme indipendenti, che anche l'E-core esegue molte per volta.

È questa la ragione per cui il rapporto di capacità della slide 5 sale con il peso della pipeline: più una pipeline passa tempo nella rete neurale, meno conta che il core sia un E-core. Le misure sono in cicli, quindi non dipendono dalla frequenza.

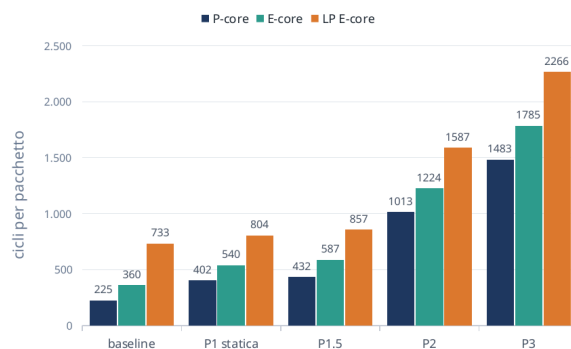
Il LP E-core paga anche la memoria

IL NUOVO ASSETTO · IL LP E-CORE

Il LP E-core paga anche la memoria



Mancate di cache. Zero sul P-core e sull'E-core; 14–19 per pacchetto sul LP E-core, fuori dalla L3.



Cicli per pacchetto. P3: 1 483 → 1 785 → 2 266; in tempo 425 → 515 → 908 ns.

QUINDI

Il LP E-core sta su un'altra parte del chip: i pacchetti scritti dal generatore gli arrivano dalla memoria. Fra memoria, IPC più basso (0,8–2,9) e 2,5 GHz regge il **36–47%** del P-core sulle pipeline.

Contatori hardware del core del nodo, per pacchetto, nodo saturo · LP E-core a 2,5 GHz, fuori dalla L3

8

A sinistra le mancate dell'ultimo livello di cache per pacchetto: quante volte il core cerca un dato nella cache più grande e non lo trova, e deve andare in memoria. Sul P-core e sull'E-core sono zero: i pacchetti che il generatore scrive restano nella cache L3, condivisa da P-core ed E-core. Il LP E-core sta fuori dalla L3: ne ha 14–19 per pacchetto, e ogni volta aspetta la memoria.

A destra i cicli per pacchetto sui tre core. Per P3: 1 483 sul P-core, 1 785 sull'E-core, 2 266 sul LP E-core. In tempo pesa anche la frequenza più bassa del LP E-core (2,5 GHz): 425, 515 e 908 ns.

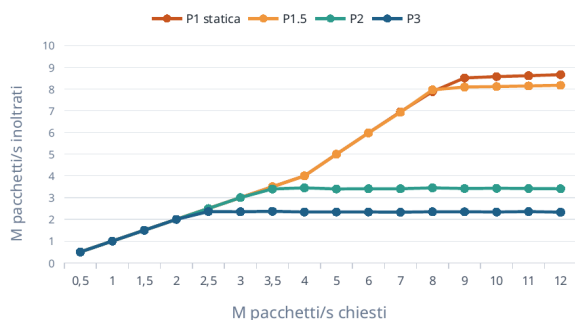
Esempio. La baseline sul LP E-core spende 733 cicli per pacchetto contro 225 sul P-core: tre volte tanto, per lo stesso codice. La differenza è quasi tutta attesa della memoria.

Il LP E-core regge il 36–47% del P-core sulle pipeline. È il caso estremo: un core pensato per tenere acceso il sistema consumando poco, non per elaborare pacchetti.

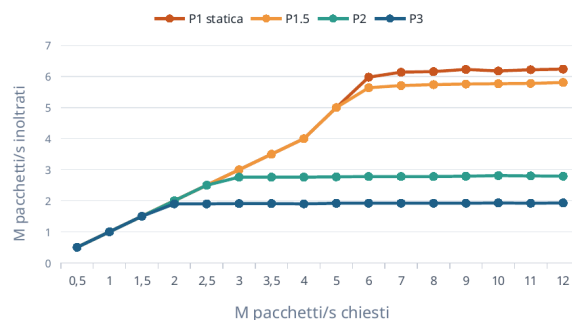
Oltre la capacità si perde all'ingresso del programma

SOTTO TRAFFICO CRESCENTE · P-CORE E E-CORE

Oltre la capacità si perde all'ingresso del programma



P-core. Ogni pipeline inoltra tutto fino alla sua capacità (P1 8,7, P2 3,4, P3 2,3 M/s), poi resta piatta: il resto è respinto dalla coda d'ingresso.



E-core. Stesso andamento, ginocchio più in basso (P1 6,2, P2 2,8, P3 1,9 M/s). A coda piena la latenza mediana sale da 13–20 μ s a 30–150 μ s.

QUINDI

I pacchetti si perdono **all'ingresso**, non dopo il programma: il collo di bottiglia è il **programma eBPF**, su ogni tipo di core. Il ginocchio coincide con la capacità misurata a massima spinta.

bench_bitrate, xdp_gen, 5 giri per punto, 64 byte · a sinistra il nodo su un P-core, a destra su un E-core

9

Qui il traffico **cresce** gradualmente, da 0,5 a 12 milioni di pacchetti al secondo, e si contano i pacchetti inoltrati. A sinistra il nodo su un P-core, a destra su un E-core.

Ogni curva sale lungo la diagonale (tutto ciò che arriva viene inoltrato) finché il traffico resta sotto la capacità della pipeline, poi diventa piatta: il nodo non può inoltrare di più, e il resto viene respinto dalla coda d'ingresso piena.

Il ginocchio delle curve coincide con la capacità misurata a massima spinta: sul P-core P1 a 8,7, P2 a 3,4, P3 a 2,3 milioni al secondo; sull'E-core P1 a 6,2, P2 a 2,8, P3 a 1,9.

Dove si perde. Il banco conta i pacchetti in tre punti: inviati, ricevuti dal programma, inoltrati. Ricevuti e inoltrati restano uguali (al massimo 0,04% di differenza): il programma rilancia tutto ciò che riceve. Tutto ciò che manca si è perso **prima** del programma, alla coda. Quindi il collo di bottiglia è il programma eBPF, non la trasmissione, su ogni tipo di core.

A coda piena il ritardo mediano sale da 13–20 μ s a 30–150 μ s: ogni pacchetto aspetta i 255 davanti a lui.

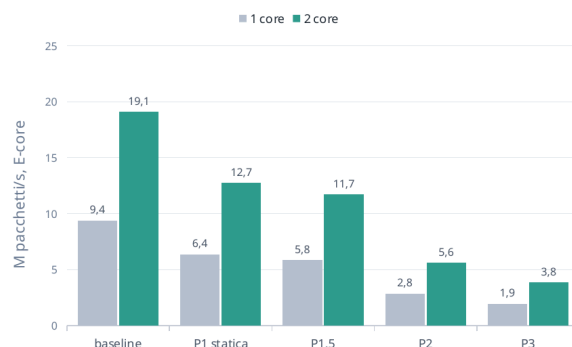
Due core reggono il doppio

IL NUOVO ASSETTO · DUE CORE

Due core reggono il doppio, sul P-core e sull'E-core



P-core. 95–102% del doppio; il costo per core resta quello di un core entro una decina di ns (P3 425 → 434).



E-core. 99–102% del doppio, con un E-core per modulo: i due moduli non si disturbano.

QUINDI

I contatori delle pipeline sono per-core e ogni core ha la sua coda d'uscita: il secondo core non rallenta il primo. Un nodo con N code regge **N volte** la capacità di un core, di qualunque tipo.

Un thread di xdp_gen per coda, una coda e un core d'uscita per core del nodo · P: CPU 6 e 8 · E: CPU 12 e 16, uno per modulo

10

Il nodo riceve **due core**, ciascuno con la sua coda d'ingresso alimentata da un thread del generatore, la sua coda d'uscita e il suo core d'uscita. A sinistra due P-core, a destra due E-core (uno per modulo). Le barre chiare sono un core, quelle scure due.

Come si legge. P2 con un P-core regge 3,45 milioni al secondo, con due 6,63: il 96% del doppio. Sull'E-core 2,83 e 5,60: il 99%.

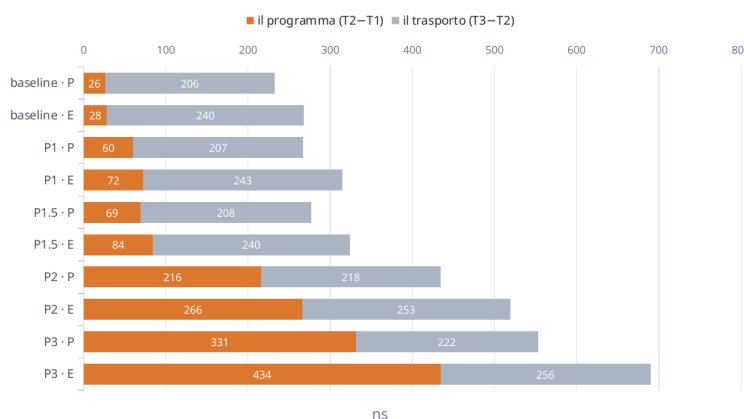
Due core reggono il 95–102% del doppio di un core, su entrambi i tipi di core, e il costo per pacchetto di ogni core resta quello di un core solo (P3: 425 ns con un core, 434 con due). Il secondo core non rallenta il primo perché non condividono niente di scritto a ogni pacchetto: i contatori delle pipeline sono per-core, e ogni core ha la sua coda d'uscita.

Conseguenza pratica: una scheda di rete vera distribuisce i pacchetti su più code e più core (RSS). Un nodo con N code regge circa N volte la capacità misurata su un core.

Tre orologi: il programma e il trasporto

DENTRO IL NODO · TRE OROLOGI · P-CORE E E-CORE

Il programma e il trasporto, su un P-core e su un E-core



Il programma

Da **26 ns** (baseline) a **331 ns** (P3) sul P-core; sull'E-core dal 7% al 31% in più (P3 434 ns).

Il trasporto

Redirect, veth e ricezione: **~210 ns** sul P-core, **~245** sull'E-core, uguale per tutte le pipeline.

QUINDI

Il trasporto è un costo comune che non dipende dalla pipeline; il programma pesa di più sull'E-core quanto più è lungo. La latenza minima arrivo → ripartenza va da 221 a 609 ns sul P-core, da 282 a 727 sull'E-core.

T1 all'ingresso del programma, T2 prima di bpf_redirect, T3 all'arrivo dall'altra parte · minimi, mediana di tre giri, da 0,5 a 3 Mpps

11

Con una versione del programma che legge l'orologio in tre punti si separa il tempo del programma da quello del trasporto:

- **T1** all'ingresso del programma, **T2** subito prima di inoltrare: T2-T1 è il programma (arancione);
- **T3** all'arrivo al nodo successivo: T3-T2 è il trasporto, cioè redirect, cavo virtuale e ricezione (grigio).

Le barre mostrano i due pezzi per ogni pipeline sul P-core (P) e sull'E-core (E).

Il trasporto è un costo comune: circa 210 ns sul P-core e 245 sull'E-core, uguale per tutte le pipeline. Il programma va da 26 ns (baseline) a 331 ns (P3) sul P-core; sull'E-core costa dal 7% al 31% in più (P3 434 ns), tanto di più quanto più è lungo.

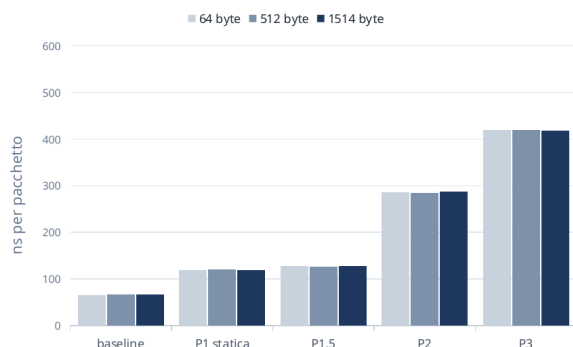
Esempio. P2 sul P-core: 216 ns di programma più 218 di trasporto. La somma, 434 ns a traffico bassissimo, è la latenza minima dall'arrivo del pacchetto alla sua ripartenza.

Una misura, non una capacità. Questa versione legge l'orologio due volte in più per pacchetto, e l'uscita resta sullo stesso core del nodo: serve a capire dove va il tempo, non a misurare quanti pacchetti regge il nodo (quello lo fanno le slide 3 e 5).

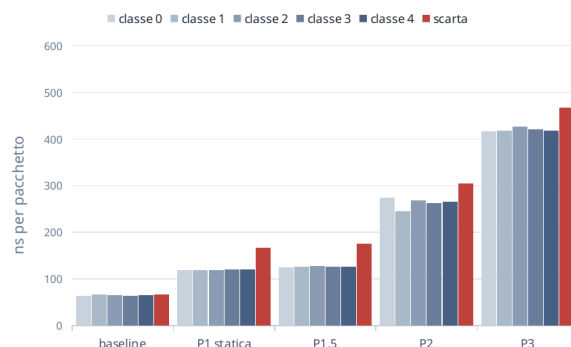
Il costo dipende dalla pipeline, e da niente altro

QUANTO È SOLIDO · P-CORE

Il costo dipende dalla pipeline, e da niente altro



Taglia del pacchetto: nessun effetto (entro l'1 %). Il nodo legge solo l'intestazione, come su una scheda di rete vera.



Porta d'uscita: nessun effetto. Scartare costa ~50 ns in più: la pagina si libera sul core del nodo.

QUINDI Basta **un numero per pipeline** a descrivere il costo del nodo: non dipende da taglia del pacchetto, porta d'uscita o TTL; solo la decisione di scartare costa ~50 ns in più. Il traffico in bit al secondo cresce quindi con la taglia del pacchetto.

Sei classi: stato dei collegamenti e TTL scelti perché la rete decida proprio quella classe; verificato su ogni pipeline · P-core a 3,5 GHz

12

Due controlli sul P-core:

- **a sinistra** la taglia del pacchetto: a 64, 512 e 1 514 byte ogni pipeline costa uguale entro l'1%. Il nodo legge solo l'intestazione, come farebbe su una scheda di rete vera;
- **a destra** la classe decisa: stato dei collegamenti e TTL sono scelti apposta perché la rete decida ciascuna delle sei classi raggiungibili. Le cinque classi che inoltrano (su porte diverse) costano uguale; la classe che **scarta** (rosso) costa circa 50 ns in più.

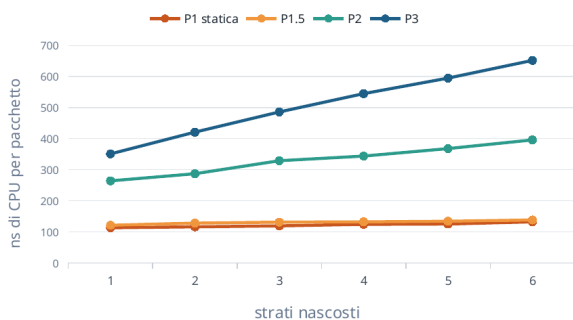
Perché scartare costa di più. Quando il pacchetto viene inoltrato, la sua memoria si libera sul core d'uscita; quando viene scartato, si libera sul core del nodo, che quindi lavora di più. È un effetto del banco, non della rete neurale.

Basta quindi un numero per pipeline a descrivere il costo del nodo: non dipende dalla taglia del pacchetto né dalla porta d'uscita. Il traffico in bit al secondo che il nodo regge cresce con la taglia del pacchetto.

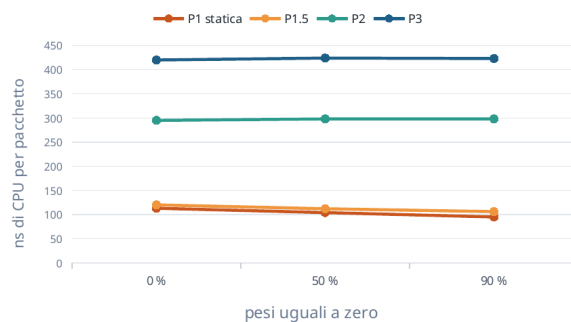
Gli stessi andamenti sul nodo vero

GLI ASSI SOTTO TRAFFICO · P-CORE

Gli stessi andamenti sul nodo vero



Profondità (strati da 4 neuroni). P3 paga **~60 ns per strato** (351 → 652 ns), P2 264 → 396; P1 e P1.5 quasi ferme.



Pesi a zero. Al 90 % di zeri P1 113 → 95 ns, P1.5 120 → 106; P2 e P3 invariate, con le stesse istruzioni per pacchetto.

QUINDI

Le conclusioni del programma da solo valgono sul nodo vero, con il **costo fisso del nodo** (~66 ns) in più. Sugli E-core le stesse curve, più alte del 20–30 %.

bench_throughput con xdp_gen, tempo di CPU per pacchetto, nodo su P-core · larghezza e pari pesi in docs/testing.md §10.8

13

Le analisi della seconda parte (slide 18–22) usano il programma da solo. Qui due di quelle analisi sono rifatte **sotto traffico vero**, con modelli sintetici che hanno lo stesso tipo di ingresso del modello reale, sul P-core.

A sinistra la profondità: strati nascosti da 4 neuroni, da 1 a 6. P3 paga circa 60 ns per strato (351 → 652 ns), P2 circa 26 (264 → 396), P1 e P1.5 quasi niente.

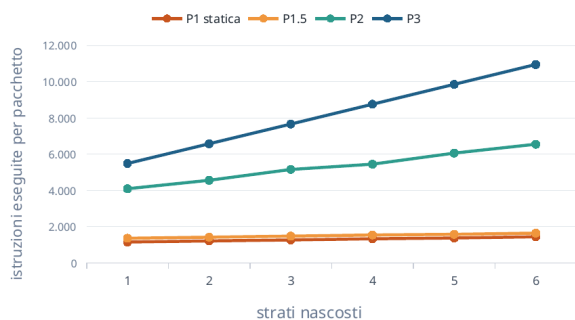
A destra i pesi a zero: lo stesso modello con lo 0, il 50 e il 90% dei pesi a zero. P1 e P1.5 accelerano (P1 113 → 95 ns, P1.5 120 → 106), perché il compilatore vede lo zero e cancella la moltiplicazione; P2 e P3 non cambiano, perché per loro uno zero letto da una tabella è un numero come un altro.

Le conclusioni del programma da solo valgono sul nodo vero, con il costo fisso del nodo (circa 66 ns) in più. Sull'E-core le stesse curve stanno più in alto del 20–30%.

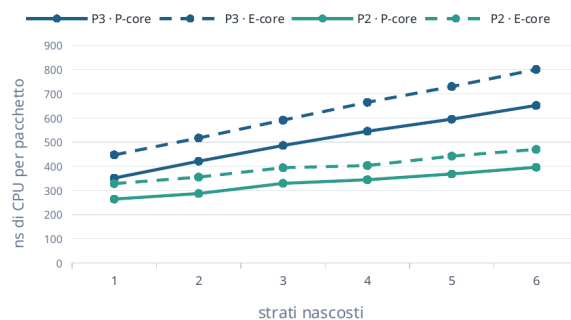
Ogni strato di P3 costa esattamente 1 096 istruzioni

GLI ASSI SOTTO TRAFFICO · LE ISTRUZIONI

Ogni strato di P3 costa esattamente 1 096 istruzioni



Istruzioni. P3: 5 488 → 10 952, **+1 096 per strato** a ogni passo, identiche su P-core ed E-core. P2 cresce a passi irregolari: srotola e ricompila.



Tempo. P3 ~60 ns per strato sul P-core (351 → 652), ~70 sull'E-core (447 → 801). Tratteggio = E-core.

QUINDI

In P3 ogni strato è un salto più le letture che ricostruiscono il contesto: sempre le stesse 1 096 istruzioni. La profondità si paga a ogni pacchetto, e sull'E-core un po' di più.

Modelli sintetici 65-4x4-7, stesso descrittore del checkpoint · contatori hardware del core del nodo, per pacchetto

14

A sinistra le istruzioni eseguite per pacchetto, dai contatori hardware, al crescere della profondità. P3 sale di **esattamente 1 096 istruzioni a ogni strato** (5 488, 6 576, 7 663, 8 759, 9 856, 10 952), identiche su P-core ed E-core. P2 cresce a passi irregolari, perché per ogni profondità ricompila e srotola un programma diverso. P1 e P1.5 crescono poco.

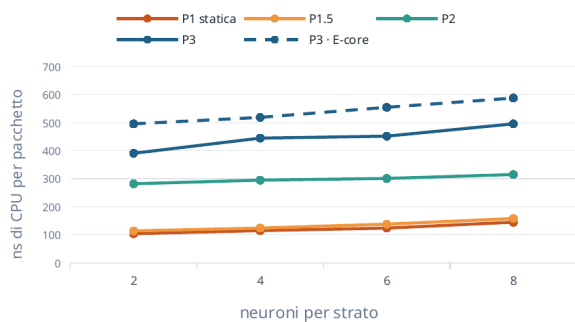
Che cosa sono le 1 096 istruzioni. P3 esegue uno strato per volta: alla fine di ogni strato salta di nuovo al programma generico dello strato, che deve rileggere da tabelle dove si trova, che forma ha lo strato e dove sono i suoi pesi. Il conto è sempre lo stesso, ed è per questo che il costo per strato è una costante.

A destra il tempo per pacchetto di P3 e P2, sul P-core (linea continua) e sull'E-core (tratteggio). P3 paga circa 60 ns per strato sul P-core e 70 sull'E-core: stesse istruzioni, eseguite a un IPC più basso.

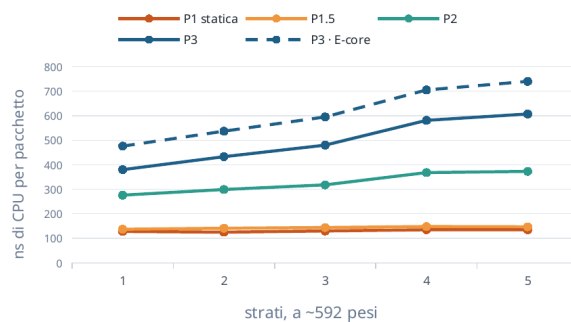
Larghezza e profondità sul nodo vero

GLI ASSI SOTTO TRAFFICO · LARGHEZZA E PARI PESI

Larghezza e profondità sul nodo vero



Larghezza. Da 2 a 8 neuroni sale su tutte: P1 104 → 145, P3 391 → 496 ns.



A pari pesi. Da 1 a 5 strati P2 +35%, P3 +60%; le P1 restano a 125-148 ns.

QUINDI

Le scelte del programma da solo valgono sul nodo vero, con il costo fisso del nodo (~66 ns) in più. Sull'E-core le stesse curve, più alte del **20-35%**: la forma migliore non cambia con il core.

Tempo di CPU per pacchetto, nodo su P-core · tratteggio: P3 sull'E-core · modelli sintetici con il descrittore del checkpoint

15

Altre due analisi sotto traffico vero, sul P-core; il tratteggio è P3 sull'E-core.

A sinistra la larghezza: neuroni per strato nascosto da 2 a 8. Sale su tutte le pipeline: P1 104 → 145 ns, P3 391 → 496. Un neurone in più è lavoro vero a ogni pacchetto.

A destra a pari pesi: cinque forme con circa 592 pesi, da uno strato largo a cinque stretti. P2 sale del 35% e P3 del 60% passando da 1 a 5 strati; P1 e P1.5 restano fra 125 e 148 ns.

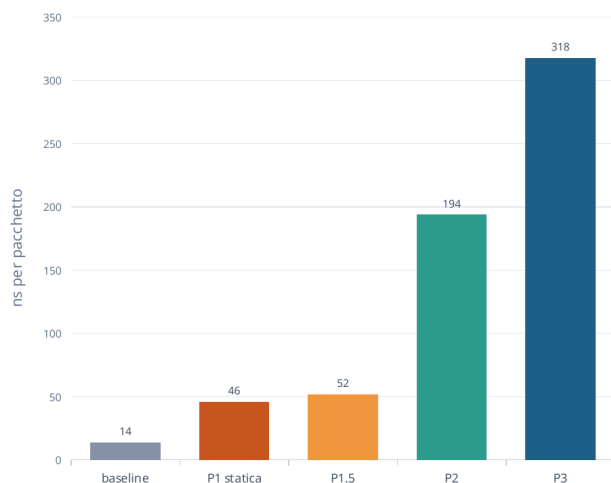
Che cosa vuol dire per chi progetta il modello. Con le pipeline specializzate (P1, P1.5) la forma della rete si sceglie solo per l'accuratezza. Con quelle generiche (P2, P3), a parità di pesi, conviene una rete larga e poco profonda.

Sull'E-core le curve hanno la stessa forma, più alte del 20-35%: la forma migliore non dipende dal core su cui girerà il nodo.

Il programma da solo: istruzioni e latenza

IL PROGRAMMA DA SOLO · ISTRUZIONI E LATENZA

La configurabilità si paga per pacchetto: 46 → 318 ns



Versione	Istruzioni	Lecture tabella	Salti
P1 statica	606	5	1
P1.5	1 019	6	1
P2	15 089	11	1
P3	12 288	29	3

QUINDI

La configurabilità si paga nelle **letture di tabella** (5 → 29) e nei **salti fra programmi** (1 → 3), non nell'aritmetica. Rendere configurabile la forma (P2) costa ~4× P1; anche la profondità (P3) ~7×. Fra P1 e P1.5 quasi niente.

Stesso modello, 319 pesi · minimo su 7 prove · BPF_PROG_TEST_RUN · P-core a 3,5 GHz

16

Da qui in poi il programma è eseguito dal kernel (BPF_PROG_TEST_RUN) su un pacchetto preparato, migliaia di volte, senza traffico: è il costo del solo programma, senza ricevere né trasmettere. Si riporta il minimo su 7 prove, sul P-core a 3,5 GHz.

Il grafico dà la latenza: dalla baseline (14 ns, nessuna rete) a P1 (46), P1.5 (52), P2 (194), P3 (318). La tabella spiega da dove viene: le **letture di tabella** per pacchetto passano da 5 a 29, i **salti fra programmi** da 1 a 3.

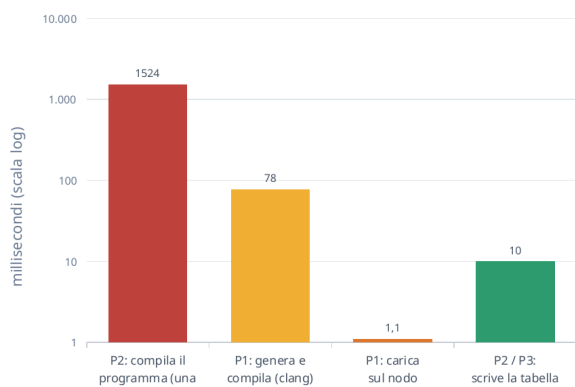
Esempio. P2 ha 15 089 istruzioni e costa 194 ns; P3 ne ha 12 288 e costa 318 ns. Il programma più piccolo è il più lento: lo spiegano i 3 salti e le 29 letture di tabella di P3 contro 1 e 11 di P2.

La configurabilità si paga nelle letture di tabella e nei salti, non nell'aritmetica. Rendere configurabile la forma (P2) costa circa 4 volte P1; anche la profondità (P3) circa 7 volte. Fra P1 e P1.5 quasi niente.

Cambiare modello costa millisecondi a tutte le pipeline

IL PROGRAMMA DA SOLO - IL COMPROMESSO

Cambiare modello costa millisecondi a tutte le pipeline



Con l'oggetto precompilato (AOT)

Il nodo carica un file pronto: **~1 ms**. Generarlo e compilarlo con clang, fuori dal nodo, costa **~80 ms**.

Il vantaggio di P2 e P3

Un modello nuovo si installa **scrivendo una tabella** (~10 ms): nessun compilatore. Il programma generico si compila una volta sola (1,2-1,5 s). P1 vuole un compilatore per ogni modello nuovo, su qualche macchina.

QUINDI Il tempo per cambiare modello **non è un criterio di scelta**: millisecondi per tutte. Si sceglie sul **costo per pacchetto** (~50 ns di rete con i pesi compilati, ~225-360 con quelli da tabella) e su **dove serve un compilatore**.

Tempo per installare un modello nuovo su un nodo già in servizio (scala logaritmica)

17

Il grafico (scala logaritmica) confronta quanto costa installare un modello nuovo su un nodo già in servizio:

- **P2: compilare il programma generico**, una volta sola all'avvio del nodo: circa 1,5 secondi;
- **P1: generare e compilare** il programma con i pesi dentro, su una macchina qualunque, fuori dal nodo: circa 80 ms;
- **P1: caricarlo sul nodo**: circa 1 ms;
- **P2 / P3: scrivere la tabella** con i pesi del modello nuovo: circa 10 ms.

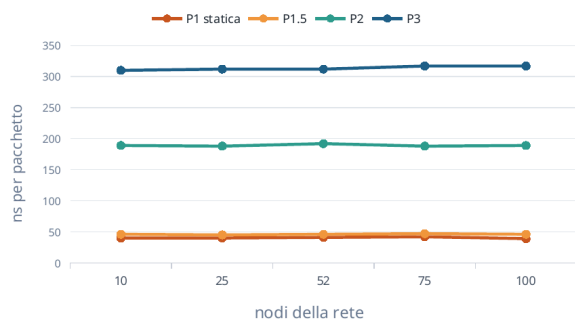
Perché P1 non ha bisogno di un compilatore sul nodo. Il programma con i pesi si compila prima (AOT, ahead of time) e al nodo arriva un file già pronto, che un piccolo caricatore apre e attacca. Il nodo non ha né clang né le librerie di sviluppo.

Il tempo per cambiare modello non è quindi un criterio di scelta: millisecondi per tutte. Si sceglie sul costo per pacchetto (circa 50 ns di rete con i pesi compilati, 225-360 con quelli da tabella) e su dove serve un compilatore: P1 ne vuole uno per ogni modello nuovo, su qualche macchina; P2 e P3 nessuno.

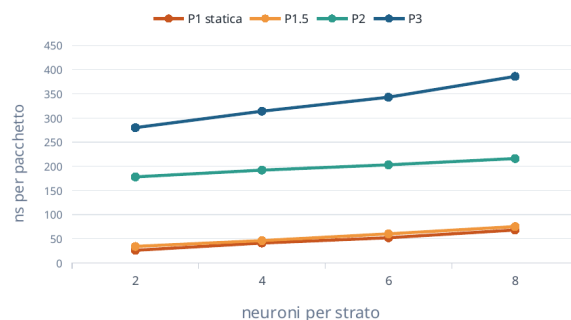
La rete può crescere quanto vuole. Il modello no.

IL PROGRAMMA DA SOLO · TAGLIA DELLA RETE, LARGHEZZA DEL MODELLO

La rete può crescere quanto vuole. Il modello no.



L'identità del nodo attiva una colonna sola di pesi. P1 statica la conosce già alla compilazione e la somma al bias; le altre la leggono da tabella. In nessun caso il lavoro per pacchetto



Da 2 a 8 neuroni sale su tutte e quattro: P1 26 → 68 ns, P3 280 → 386 ns. Un neurone in più è lavoro vero, a ogni pacchetto.

QUINDI

Una rete più grande (più nodi) non rallenta nessuna pipeline: basta una riga in più in tabella. Un modello più largo rallenta tutte, a ogni pacchetto. Si può far crescere la rete; il modello va tenuto piccolo.

BPF_PROG_TEST_RUN, minimo su 7 prove · P1 come oggetto AOT · P-core a 3,5 GHz

18

A sinistra la taglia della rete: da 10 a 100 nodi. L'identità del nodo è un ingresso con un solo 1 (one-hot) e attiva una sola colonna di pesi, qualunque sia la sua larghezza: nessuna pipeline rallenta. P1 statica conosce già il nodo quando viene compilata, le altre lo leggono da tabella.

A destra la larghezza del modello: da 2 a 8 neuroni per strato. Sale su tutte e quattro: P1 26 → 68 ns, P3 280 → 386 ns. Un neurone in più è lavoro vero a ogni pacchetto.

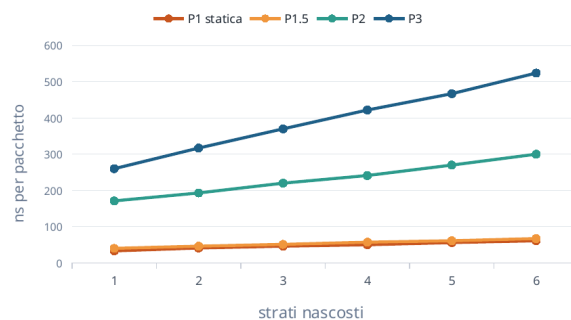
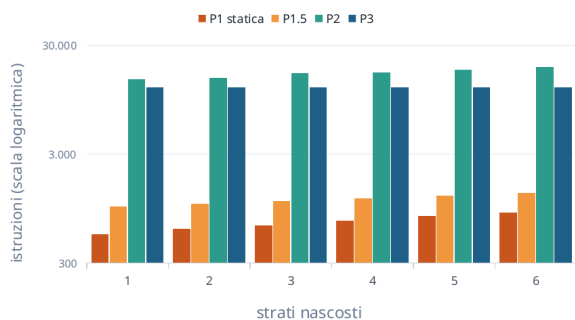
Esempio. Una rete di 100 nodi costa a P3 317 ns, una di 10 nodi 310: la stessa cosa. Un modello con 8 neuroni per strato invece di 4 costa a P3 386 ns invece di 314.

Una rete più grande (più nodi) non rallenta nessuna pipeline: basta una riga in più in tabella. Un modello più largo rallenta tutte. Si può far crescere la rete; il modello va tenuto piccolo.

Uno strato in più: P2 lo paga in spazio, P3 in tempo

IL PROGRAMMA DA SOLO · PROFONDITÀ

Uno strato in più: P2 lo paga in spazio, P3 in tempo



Dimensione. P2 ricompila e srotola: 14 604 → 18 937 istruzioni. P3 non si muove: 12 288 a ogni profondità.

Tempo. P3 paga: 260 → 524 ns da 1 a 6 strati, perché ogni strato è un salto fra programmi (da 2 a 7).

QUINDI

Un **modello profondo** si paga sempre: con P2 in un programma più grande da ricompilare (~870 istruzioni per strato), con P3 in **~53 ns per pacchetto a ogni strato** (~60 sotto traffico). P3 è la più flessibile, ed è quella che la profondità rallenta di più.

Quattro neuroni per strato, da 1 a 6 strati · il resto del modello fermo

19

Quattro neuroni per strato, da 1 a 6 strati, tutto il resto fermo.

A sinistra la dimensione (scala logaritmica): P2 ricompila e srotola per ogni profondità, e cresce da 14 604 a 18 937 istruzioni, circa 870 per strato. P3 non si muove: 12 288 istruzioni a ogni profondità, perché esegue sempre lo stesso strato generico.

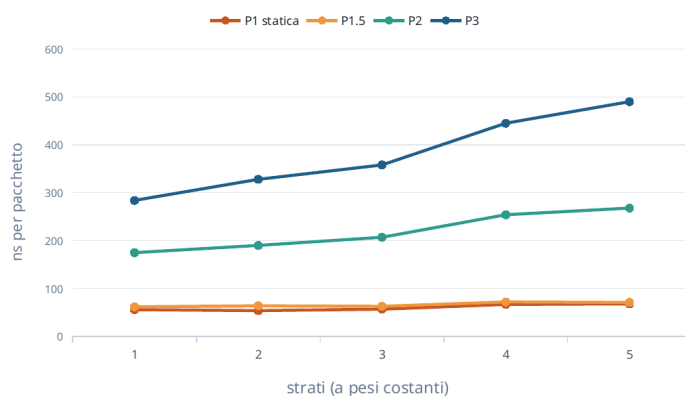
A destra il tempo: P3 paga 260 → 524 ns da 1 a 6 strati, circa 53 ns per strato, perché ogni strato è un salto fra programmi e un giro di letture di tabella (sotto traffico circa 60 ns, slide 14). P2 paga circa 26 ns per strato.

Un modello profondo si paga sempre: con P2 in un programma più grande da ricompilare, con P3 in tempo a ogni pacchetto. P3 è la più flessibile, ed è quella che la profondità rallenta di più.

«Meglio larga o profonda?» dipende da chi la esegue

IL PROGRAMMA DA SOLO - A PARITÀ DI PESI

«Meglio larga o profonda?» dipende da chi la esegue



Generiche: la profondità si paga

Da 1 a 5 strati, a pesi fermi: **P2 175 → 268 ns**, **P3 284 → 490 ns**.

Specializzate: quasi indifferenti

54-72 ns: srotolano tutto in una funzione sola, e la forma conta poco.

QUINDI

Con le versioni **specializzate** la forma della rete si sceglie per l'**accuratezza**. Con le **generiche**, a parità di pesi, conviene una rete **larga e poco profonda**: da 1 a 5 strati la latenza sale del 53 % (P2) e del 73 % (P3).

Cinque forme con circa 592 pesi: uno strato largo, oppure cinque strati

20

Cinque forme con circa 592 pesi: uno strato largo da 8 neuroni, oppure fino a cinque strati più stretti. Lo stesso numero di pesi, disposto in modo diverso.

- **Generiche (P2, P3): la profondità si paga.** Da 1 a 5 strati P2 passa da 175 a 268 ns (+53%), P3 da 284 a 490 ns (+73%).
- **Specializzate (P1, P1.5): quasi indifferenti.** 54-72 ns senza un andamento chiaro: il compilatore srotola tutto in una funzione sola, e la forma conta poco.

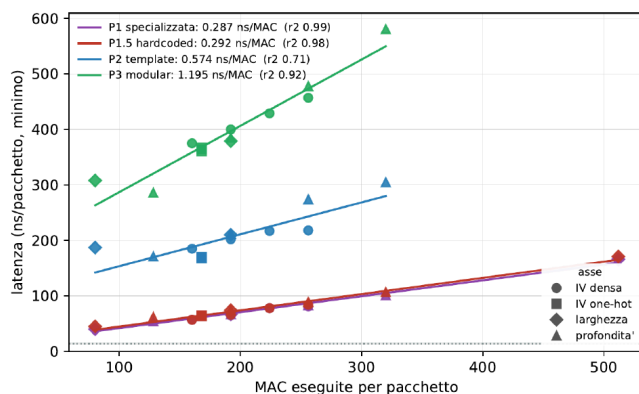
Esempio. Lo stesso budget di 592 pesi in un solo strato costa a P3 284 ns, in cinque strati 490: la scelta della forma vale 200 ns per pacchetto con P3, una decina con P1.

Con le versioni specializzate la forma della rete si sceglie per l'accuratezza. Con le generiche, a parità di pesi, conviene una rete larga e poco profonda. Sotto traffico vero lo stesso (slide 15).

Il costo per moltiplicazione è una costante della pipeline

IL PROGRAMMA DA SOLO · CONTARE IL LAVORO GIUSTO

Il costo per moltiplicazione è una costante della pipeline



Versione	ns per moltiplicazione
P1 statica	0,29
P1.5	0,29
P2	0,57
P3	1,20

Contando le moltiplicazioni **davvero eseguite**, quattro esperimenti diversi cadono sulla stessa retta (r² 0,92–0,99; P2 0,71). Un ingresso «a una casella accesa» ne attiva **una** colonna sola.

QUINDI Il costo di un modello si **stima prima di misurarlo**: moltiplicazioni eseguite × ~0,3 ns (pesi compilati) o × ~0,6–1,2 ns (pesi da tabella), più il costo fisso della pipeline. Una feature one-hot conta una colonna, qualunque sia la sua larghezza.

Contando le moltiplicazioni dalla forma del modello, per P3 la retta spiega il 10 % dei punti; contando quelle eseguite, il 92 %

21

Si può prevedere quanto costerà un modello prima di misurarlo? Il grafico mette in relazione la latenza con il numero di moltiplicazioni per pacchetto, su quattro esperimenti diversi (ingressi densi, ingressi one-hot, larghezza, profondità).

Il trucco è contare le moltiplicazioni **davvero eseguite**. Un ingresso one-hot occupa molte colonne nella matrice dei pesi ma ne attiva una sola: contato dalla forma del modello sembra costoso, nel programma no. Contando quelle eseguite, i quattro esperimenti cadono sulla stessa retta, una per pipeline (r² 0,92–0,99; P2 0,71).

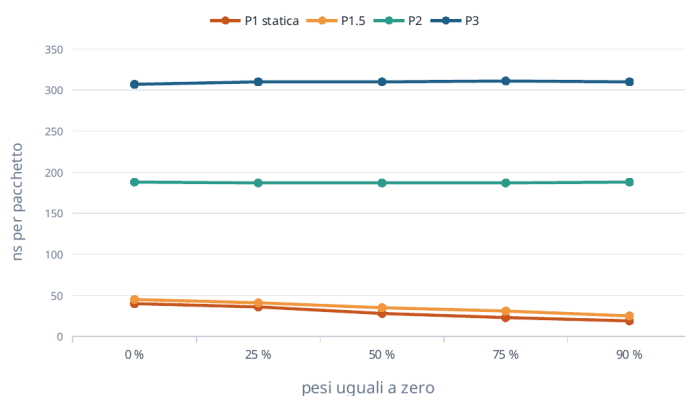
Esempio. Contando le moltiplicazioni dalla forma, per P3 la retta spiega solo il 10% dei punti; contando quelle eseguite, il 92%.

La pendenza è il costo di una moltiplicazione: circa 0,29 ns con i pesi compilati (P1, P1.5), 0,57 ns per P2, 1,20 ns per P3, che legge ogni peso da una tabella. Il costo di un modello si stima quindi come moltiplicazioni eseguite × costo per moltiplicazione, più il costo fisso della pipeline.

Pesi a zero: solo le specializzate dimagriscono

IL PROGRAMMA DA SOLO · QUANDO LA SPECIALIZZAZIONE PAGA DAVVERO

Pesi a zero: solo le specializzate dimagriscono



P1.5, da 0 a 90 % di zeri

1 045 → 337 istruzioni, 45 → 25 ns. Il compilatore vede lo zero e cancella la moltiplicazione.

P2 e P3, stessi pesi

Dimensione e latenza invariate: zero, letto da una tabella, è un numero come un altro.

QUINDI

Potare il modello (pesi a zero) rende più veloce solo le versioni specializzate: al 90 % di zeri P1.5 perde il 44 % della latenza e due terzi delle istruzioni. Con P2 e P3 non dà niente.

È l'unico esperimento in cui scendono insieme dimensione e tempo · sotto traffico P1.5 passa da 120 a 106 ns

22

Lo stesso modello con una frazione crescente di pesi esattamente a zero, da 0 al 90%.

- **P1 e P1.5** scendono: P1.5 passa da 1 045 a 337 istruzioni e da 45 a 25 ns. Il compilatore vede che il peso è zero e cancella la moltiplicazione.
- **P2 e P3** non cambiano né in dimensione né in tempo: uno zero letto da una tabella è un numero come un altro, e la moltiplicazione si fa comunque.

Sotto traffico vero lo stesso. Al 90% di zeri P1.5 passa da 120 a 106 ns di CPU per pacchetto; P2 e P3 restano fermi (slide 13).

È l'unico esperimento in cui dimensione e tempo scendono insieme. Potare il modello (mettere pesi a zero) rende più veloci solo le versioni specializzate; con P2 e P3 la potatura non dà niente in velocità, e va giustificata solo dall'accuratezza.